

**DESIGN AND SIMULATE SMART ELEVATOR AUTOMATION AND
HUMAN DETECTION SYSTEM USING PIR
SENSOR. THE SYSTEM AUTOMATICALLY DETECTS HUMAN
PRESENCE AND CONTROLS ELEVATOR OPERATIONS FOR
IMPROVED SAFETY AND AUTOMATION.**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B	– 24MER030
MOHAMMED SABEER S	– 24MER031
PRAAVIN RAJAA G	– 24MER039
PRASANNA T	– 24MER042
DEVA PRIYAN R	– 24MEL064
SATHISH KUMAR K	– 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Smart Elevator Automation and Human Detection System using PIR Sensor is designed to improve the safety, efficiency, and automation of elevator systems. In conventional elevators, most operations depend on manual inputs such as push buttons for calling the elevator and selecting floors. The proposed system introduces automatic human detection to make elevator operation more intelligent and responsive.

The system uses a PIR (Passive Infrared) sensor to detect human presence or movement near the elevator. The PIR sensor senses the infrared radiation emitted by the human body and generates a signal when a person enters its detection range. This signal is given to a microcontroller, such as Arduino or ESP32, which acts as the main control unit.

Based on the detected human presence and user commands, the controller operates the elevator motor and door mechanism. The system can automatically open the elevator door when a person is detected, control the elevator movement according to the selected floor, and close the door after the passenger enters or exits.

The system also provides status indications using LEDs, a buzzer, and an LCD/OLED display. These components can indicate the current floor, elevator movement, door status, and warning conditions. When no human presence is detected, the elevator can enter a standby condition to reduce unnecessary operation and energy consumption.

The project is developed and simulated to demonstrate the basic working of an intelligent elevator system. It provides a low-cost and simple approach for integrating human detection, automatic control, and safety features into elevator applications.

2. Problem Statement

Traditional elevator systems mainly depend on manual operation. A passenger generally needs to press a call button, enter the elevator, select a floor, and operate the system through different controls. Although modern elevators have several safety features, there is still a need for improved automatic human detection and intelligent operation.

One of the important problems is that the elevator may operate unnecessarily when there is no person waiting or using the elevator. This can result in unnecessary energy consumption and increased mechanical operation.

Another important issue is **door safety**. If the elevator door starts closing while a person is entering or exiting, there is a possibility of accidental contact or injury. A human detection system can provide additional information to the controller so that the door operation can be controlled more safely.

The lack of automatic human presence detection can also make the elevator less efficient in situations where passengers enter or leave quickly. Therefore, a system is required that can

detect human presence and use the detected information to control elevator operations automatically.

The major problems addressed by this project are:

- Lack of automatic human presence detection.
- Unnecessary elevator operation.
- Increased energy consumption.
- Possible door-closing safety issues.
- Dependence on manual operation.
- Limited intelligent control of elevator functions.
- Need for a simple and economical automation system.

3. Proposed Solution

The proposed solution is to develop a **Smart Elevator Automation and Human Detection System using a PIR sensor**. The PIR sensor is positioned near the elevator entrance or inside the elevator model to detect human movement or presence.

When a person enters the detection area, the PIR sensor detects the change in infrared radiation and sends a signal to the microcontroller. The microcontroller processes this signal and determines the required elevator action.

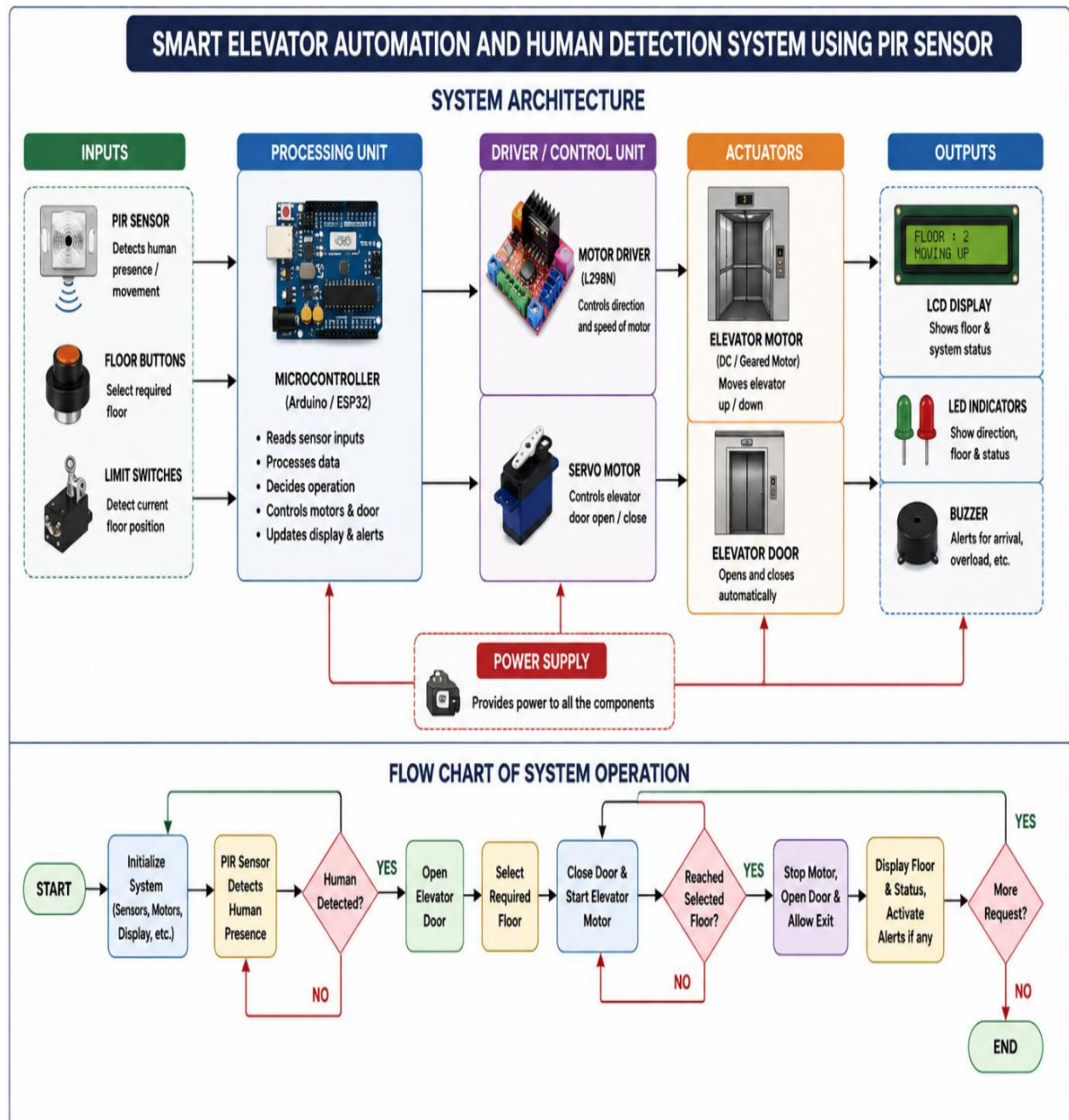
The controller can operate a **servo motor or DC motor** to demonstrate elevator door movement and a **DC/stepper motor** to represent elevator cabin movement. Push buttons can be provided for selecting the required floor. The controller coordinates the sensor input, floor selection, door operation, and motor operation.

The proposed system can work as follows:

1. The system is switched ON and all components are initialized.
2. The PIR sensor continuously monitors the elevator area.
3. When human presence is detected, the sensor sends a signal to the microcontroller.
4. The controller activates the elevator door mechanism.
5. The passenger selects the required floor using a push button.
6. The controller activates the elevator motor.
7. The elevator moves toward the selected floor.
8. The controller stops the motor when the required floor is reached.
9. The elevator door opens automatically.
10. After a predefined time, the door closes.

4. System Architecture

The overall architecture of the system is:

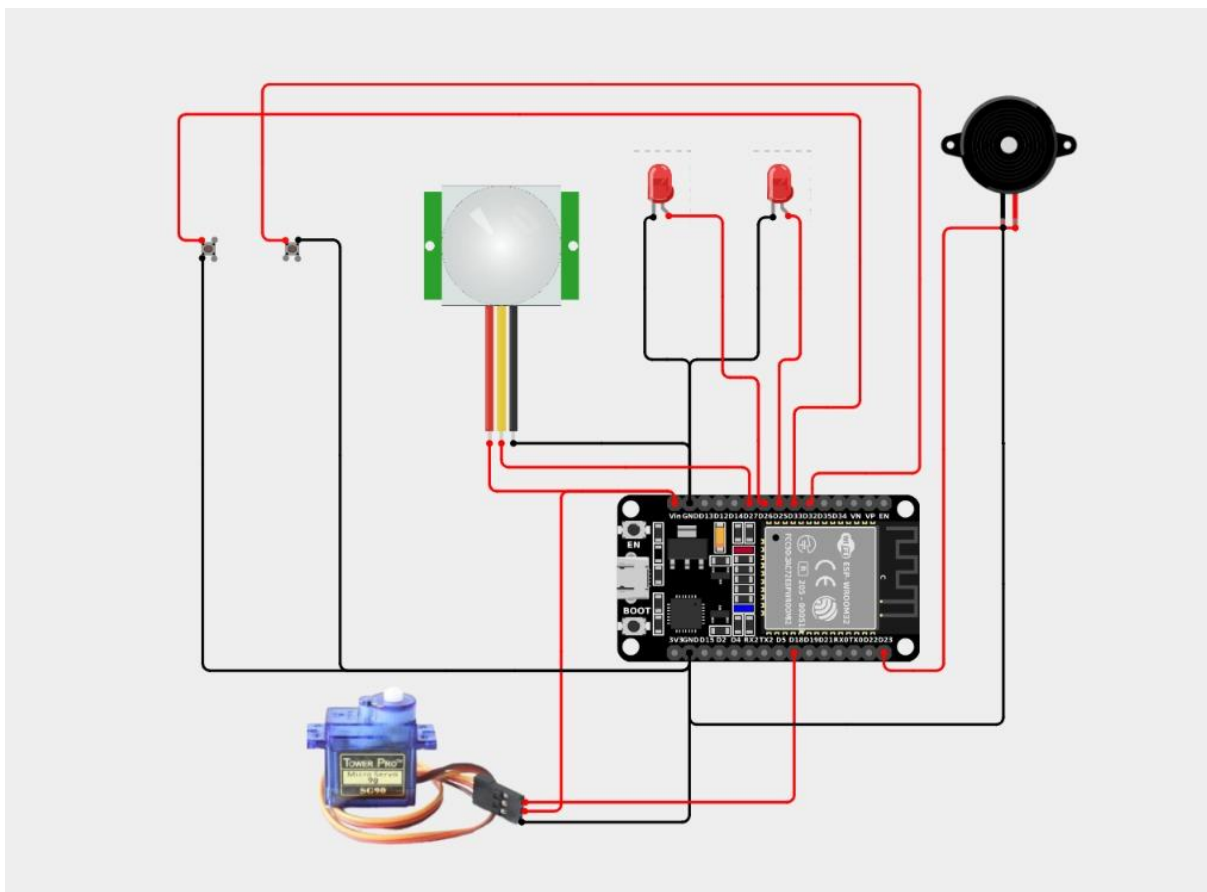


5. Circuit Table

S.No.	Component	Pin Terminal	Connected To	Purpose
1	PIR Sensor	VCC	Arduino 5V	Power supply
		GND	Arduino GND	Ground
		OUT	Digital Pin D2	Human detection signal
2	Floor Selection Buttons	Button 1	D3	Select Floor 1
		Button 2	D4	Select Floor 2
		Button 3	D5	Select Floor 3
3	Limit Switches	Floor 1	D6	Detect elevator position
		Floor 2	D7	Detect elevator position
		Floor 3	D8	Detect elevator position
4	Motor Driver (L298N)	IN1	D9	Motor direction
		IN2	D10	Motor direction
		ENA	D11	Motor speed control
5	Elevator Motor	Motor terminals	L298N OUT1/OUT2	Moves elevator cabin
6	Servo Motor	Signal	D12	Opens/closes elevator door
		VCC	5V	Power supply
		GND	GND	Ground
7	LCD Display	SDA	A4	Display communication
		SCL	A5	Display communication
8	Green LED	Anode	D13 through resistor	Safe/normal indication

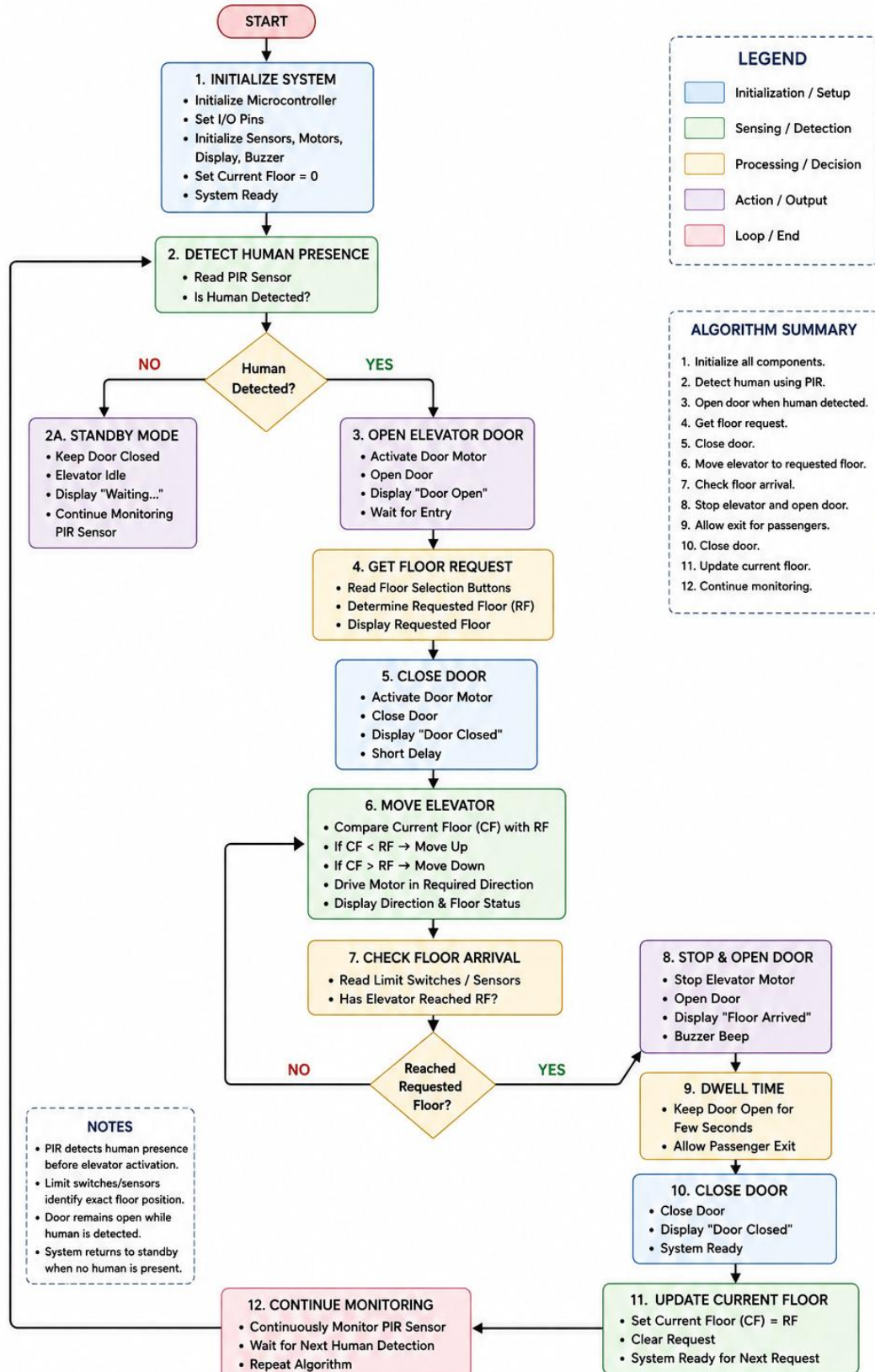
S.No.	Component	Pin Terminal /	Connected To	Purpose
9	Red LED	Anode	A0 through resistor	Warning/operation indication
10	Buzzer	Positive	A1	Audio alert
		Negative	GND	Ground
11	Arduino/ESP32	5V/VIN	Power Supply	Main power
		GND	Common GND	Common ground

6. Circuit Design



7. Algorithm

SMART ELEVATOR AUTOMATION AND HUMAN DETECTION SYSTEM WORKING ALGORITHM



8. Coding

```
#include <WiFi.h>

#include <HTTPClient.h>

// =====

// WIFI SETTINGS

// =====

const char* WIFI_SSID   = "HITMAN";
const char* WIFI_PASSWORD = "123456789";

// =====

// THINGSPEAK SETTINGS

// =====

const char* THINGSPEAK_API_KEY = "XXC6TULBUG960CWH";

const char* THINGSPEAK_SERVER =
    "http://api.thingspeak.com/update";

// =====

// ESP32 PIN CONNECTIONS

// =====

#define PIR_PIN    27
#define SERVO_PIN  13

#define LED1_PIN   14
#define LED2_PIN   12
```



```
#define BUZZER_PIN 26
```

```
#define BUTTON1_PIN 32
```

```
#define BUTTON2_PIN 33
```

```
// =====
```

```
// SERVO SETTINGS
```

```
// =====
```

```
#define SERVO_CLOSED 0
```

```
#define SERVO_OPEN 90
```

```
int servoPosition = SERVO_CLOSED;
```

```
// =====
```

```
// VARIABLES
```

```
// =====
```

```
int pirStatus = 0;
```

```
int led1Status = 0;
```

```
int led2Status = 0;
```

```
int buzzerStatus = 0;
```

```
int button1Status = 0;
```

```
int button2Status = 0;
```

```
int operatingMode = 0;
```

```
// 0 = Automatic
```

```
// 1 = Manual
```

```
bool manualMode = false;
```

```
unsigned long lastThingSpeakUpdate = 0;
```

```
// ThingSpeak requires an interval between updates
```

```
const unsigned long THINGSPEAK_INTERVAL = 16000;
```

```
// =====
```

```
// SERVO PWM FUNCTION
```

```
// =====
```

```
void setServoAngle(int angle)
```

```
{
```

```
    int pulseWidth;
```

```
    // Convert angle 0-180 to pulse width
```

```
    pulseWidth = map(angle, 0, 180, 500, 2400);
```

```
    // 50 Hz servo signal
```

```
    int duty = map(pulseWidth, 0, 20000, 0, 65535);
```

```
    ledcWrite(SERVO_PIN, duty);
```

```
    servoPosition = angle;
```

```
}
```

```
// =====
```

```
// BUZZER FUNCTION
```

```
// =====
```

```
void buzzerBeep()
```

```
{
```

```
    digitalWrite(BUZZER_PIN, HIGH);
```

```
    buzzerStatus = 1;
```

```
    delay(200);
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
    buzzerStatus = 0;
```

```
}
```

```
// =====
```

```
// OPEN SERVO
```

```
// =====
```

```
void openDoor()
```

```
{
```

```
    setServoAngle(SERVO_OPEN);
```

```
    digitalWrite(LED1_PIN, HIGH);
```

```
    led1Status = 1;
```

```
    Serial.println("SERVO: OPEN");
```

```
}
```

```
// =====
```

```
// CLOSE SERVO
```

```
// =====
```

```
void closeDoor()
```

```
{
```

```
  setServoAngle(SERVO_CLOSED);
```

```
  digitalWrite(LED1_PIN, LOW);
```

```
  led1Status = 0;
```

```
  Serial.println("SERVO: CLOSED");
```

```
}
```

```
// =====
```

```
// WIFI CONNECTION
```

```
// =====
```

```
void connectWiFi()
```

```
{
```

```
  Serial.println();
```

```
  Serial.println("Connecting to WiFi...");
```

```
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
  int attempts = 0;
```

```
  while (WiFi.status() != WL_CONNECTED && attempts < 30)
```

```
  {
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
    attempts++;  
}  
  
Serial.println();  
  
if (WiFi.status() == WL_CONNECTED)  
{  
    Serial.println("WiFi Connected!");  
    Serial.print("IP Address: ");  
    Serial.println(WiFi.localIP());  
}  
else  
{  
    Serial.println("WiFi Connection Failed!");  
}  
}  
  
// =====  
// SEND DATA TO THINGSPEAK  
// =====  
  
void sendToThingSpeak()  
{  
    if (WiFi.status() != WL_CONNECTED)  
    {  
        Serial.println("WiFi disconnected.");  
        connectWiFi();  
    }  
}
```

```
if (WiFi.status() == WL_CONNECTED)
{
    HTTPClient http;

    String url = String(THINGSPEAK_SERVER);

    url += "?api_key=";
    url += THINGSPEAK_API_KEY;

    url += "&field1=";
    url += String(pirStatus);

    url += "&field2=";
    url += String(servoPosition);

    url += "&field3=";
    url += String(led1Status);

    url += "&field4=";
    url += String(led2Status);

    url += "&field5=";
    url += String(buzzerStatus);

    url += "&field6=";
    url += String(button1Status);

    url += "&field7=";
    url += String(button2Status);
```

```
url += "&field8=";
url += String(operatingMode);

Serial.println();
Serial.println("Sending data to ThingSpeak...");
Serial.println(url);

http.begin(url);

int httpCode = http.GET();

if (httpCode > 0)
{
    Serial.print("ThingSpeak Response: ");
    Serial.println(httpCode);

    String response = http.getString();

    Serial.print("Entry ID: ");
    Serial.println(response);
}
else
{
    Serial.print("ThingSpeak Error: ");
    Serial.println(http.errorToString(httpCode));
}

http.end();
```

```

    }
}

// =====

// PRINT STATUS

// =====

void printStatus()
{
    Serial.println();
    Serial.println("=====");

    Serial.print("PIR Motion    : ");

    if (pirStatus == 1)
        Serial.println("MOTION DETECTED");
    else
        Serial.println("NO MOTION");

    Serial.print("Servo Position  : ");
    Serial.print(servoPosition);
    Serial.println(" degrees");

    Serial.print("LED 1        : ");
    Serial.println(led1Status ? "ON" : "OFF");

    Serial.print("LED 2        : ");
    Serial.println(led2Status ? "ON" : "OFF");

```



```
Serial.print("Buzzer      : ");  
Serial.println(buzzerStatus ? "ON" : "OFF");
```

```
Serial.print("Button 1    : ");  
Serial.println(button1Status ? "PRESSED" : "NOT PRESSED");
```

```
Serial.print("Button 2    : ");  
Serial.println(button2Status ? "PRESSED" : "NOT PRESSED");
```

```
Serial.print("Operating Mode : ");
```

```
if (operatingMode == 0)  
    Serial.println("AUTOMATIC");  
else  
    Serial.println("MANUAL");
```

```
Serial.println("=====");  
}
```

```
// =====  
// SETUP  
// =====
```

```
void setup()  
{  
    Serial.begin(115200);
```

```
// -----  
// PIN MODES
```

```
// -----  
  
pinMode(PIR_PIN, INPUT);  
  
pinMode(LED1_PIN, OUTPUT);  
pinMode(LED2_PIN, OUTPUT);  
  
pinMode(BUZZER_PIN, OUTPUT);  
  
// Push buttons connected to GND  
pinMode(BUTTON1_PIN, INPUT_PULLUP);  
pinMode(BUTTON2_PIN, INPUT_PULLUP);  
  
// -----  
// SERVO PWM  
// -----  
  
// ESP32 LEDC PWM  
ledcAttach(SERVO_PIN, 50, 16);  
  
setServoAngle(SERVO_CLOSED);  
  
// -----  
// INITIAL OUTPUTS  
// -----  
  
digitalWrite(LED1_PIN, LOW);  
digitalWrite(LED2_PIN, LOW);  
digitalWrite(BUZZER_PIN, LOW);
```

```
// -----  
// START MESSAGE  
// -----
```

```
Serial.println();  
Serial.println("=====");  
Serial.println(" ESP32 SMART MOTION MONITORING SYSTEM");  
Serial.println("=====");
```

```
Serial.println("System Starting...");
```

```
// -----  
// WIFI  
// -----
```

```
connectWiFi();
```

```
delay(1000);
```

```
Serial.println("System Ready!");  
}
```

```
// =====  
// LOOP  
// =====
```

```
void loop()  
{
```

```
// =====  
// READ SENSORS  
// =====  
  
pirStatus = digitalRead(PIR_PIN);  
  
// Buttons are active LOW  
button1Status = (digitalRead(BUTTON1_PIN) == LOW) ? 1 : 0;  
  
button2Status = (digitalRead(BUTTON2_PIN) == LOW) ? 1 : 0;  
  
// =====  
// BUTTON 1 - OPEN  
// =====  
  
if (button1Status == 1)  
{  
    Serial.println();  
    Serial.println("BUTTON 1 PRESSED");  
    Serial.println("Manual OPEN");  
  
    manualMode = true;  
    operatingMode = 1;  
  
    digitalWrite(LED2_PIN, HIGH);  
    led2Status = 1;  
  
    openDoor();  
}
```

```
buzzerBeep();

delay(500);
}

// =====
// BUTTON 2 - CLOSE
// =====

if (button2Status == 1)
{
    Serial.println();
    Serial.println("BUTTON 2 PRESSED");
    Serial.println("Manual CLOSE");

    manualMode = false;
    operatingMode = 0;

    digitalWrite(LED2_PIN, LOW);
    led2Status = 0;

    closeDoor();

    buzzerBeep();

    delay(500);
}

// =====
```

```
// AUTOMATIC PIR MODE
```

```
// =====
```

```
if (!manualMode)
```

```
{
```

```
    operatingMode = 0;
```

```
    if (pirStatus == 1)
```

```
    {
```

```
        Serial.println();
```

```
        Serial.println("MOTION DETECTED!");
```

```
        Serial.println("Automatic Door Opening");
```

```
        openDoor();
```

```
        buzzerBeep();
```

```
        delay(1000);
```

```
    }
```

```
    else
```

```
    {
```

```
        Serial.println("No Motion");
```

```
        closeDoor();
```

```
    }
```

```
}
```

```
// =====
```

```
// SERIAL MONITOR
```

```
// =====  
  
printStats();  
  
// =====  
// THINGSPEAK UPDATE  
// =====  
  
if (millis() - lastThingSpeakUpdate >= THINGSPEAK_INTERVAL)  
{  
    lastThingSpeakUpdate = millis();  
  
    sendToThingSpeak();  
}  
  
delay(500);  
}
```

9. System Working

The Smart Elevator Automation and Human Detection System works by detecting human presence using a PIR sensor and controlling the elevator automatically through a microcontroller.

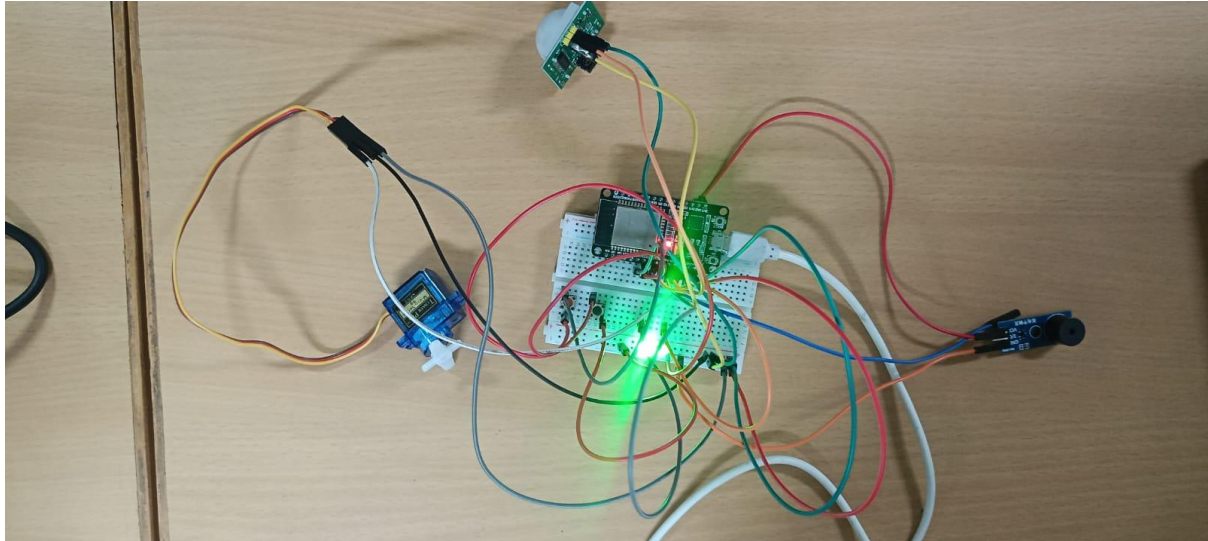
1. **System Initialization:** The Arduino/ESP32 initializes the PIR sensor, floor buttons, motor driver, door motor, display, and buzzer.
2. **Human Detection:** The PIR sensor continuously monitors the elevator area. When a person is detected, it sends a signal to the microcontroller.
3. **Door Operation:** The controller activates the servo motor to open the elevator door and allows the passenger to enter.
4. **Floor Selection:** The passenger selects the required floor using the floor selection buttons.
5. **Elevator Movement:** The microcontroller compares the current floor with the selected floor and controls the elevator motor to move up or down.
6. **Floor Detection:** Limit switches or position sensors detect when the elevator reaches the selected floor.
7. **Stopping:** When the required floor is reached, the controller stops the elevator motor.
8. **Door Opening:** The elevator door automatically opens, and the buzzer/display indicates that the selected floor has been reached.
9. **Door Closing:** After a short delay, the door closes automatically.
10. **Continuous Monitoring:** The system returns to monitoring mode and waits for the next person or floor request.

10. Bill of Materials

S.No.	Component	Specification	Quantity	Purpose
1	Arduino UNO	ATmega328P	1	Main controller
2	PIR Sensor	HC-SR501	1	Detects human presence
3	DC Geared Motor	5–12 V	1	Moves elevator cabin
4	Motor Driver	L298N	1	Controls motor direction
5	Servo Motor	SG90	1	Opens and closes elevator door
6	Push Buttons	5 mm / 6 mm	3	Floor selection
7	Limit Switch	Mechanical	3	Detects floor position
8	LCD Display	16×2 I2C	1	Displays floor and status
9	Buzzer	5 V	1	Provides alert sound
10	LED	Red & Green	2	Indicates system status
11	Resistors	220 Ω	2	LED protection
12	Breadboard	Standard	1	Circuit assembly
13	Jumper Wires	Male–Male/Female	1 set	Electrical connections
14	Power Supply	5–12 V DC	1	Provides power
15	Elevator Model/Frame	Acrylic or cardboard	1	Physical elevator structure
16	Pulley & Thread/Belt	Suitable for model	1 set	Transfers motor motion

11. Prototype Implementation

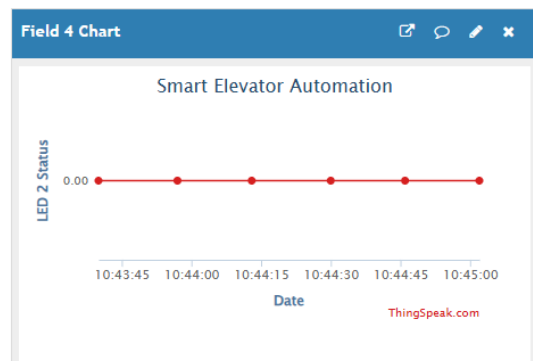
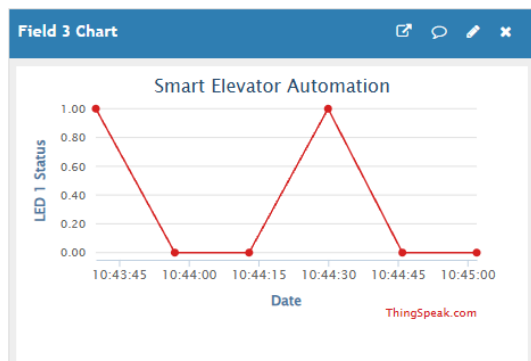
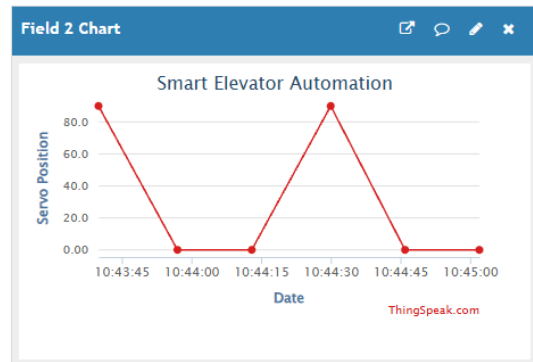
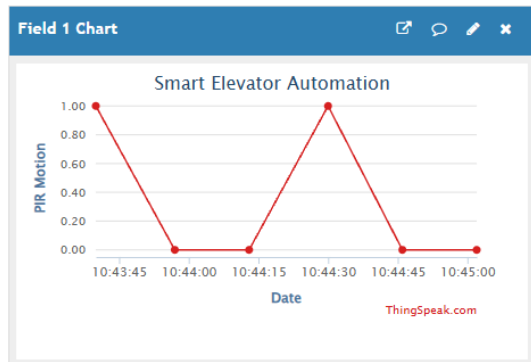
The prototype is built using an Arduino UNO, PIR sensor, DC motor, L298N motor driver, servo motor, push buttons, limit switches, LCD, LED, and buzzer. The PIR sensor detects human presence and sends the signal to the Arduino. The Arduino controls the elevator door and motor according to the selected floor. Limit switches detect the floor position, while the LCD and buzzer indicate the elevator status.



12. Results & Performance Analysis

The experimental results show that the Smart Elevator Automation System successfully detected human presence and responded automatically.

- **PIR Motion:** The PIR sensor detected human movement at approximately 10:43:45 and 10:44:30.
- **Servo Position:** The servo moved to about 90° when human presence was detected, indicating door opening, and returned to 0° when the person was not detected.
- **LED 1 Status:** LED 1 followed the PIR sensor signal, turning ON during human detection and OFF otherwise.
- **LED 2 Status:** LED 2 remained OFF (0) during the recorded period.
- **Response:** The graphs show that the system responded consistently to human detection and controlled the elevator door automatically.



Overall Performance

The results confirm that the prototype provides reliable human detection, automatic door control, and clear status indication. The system demonstrates improved safety, automation, and efficient elevator operation.